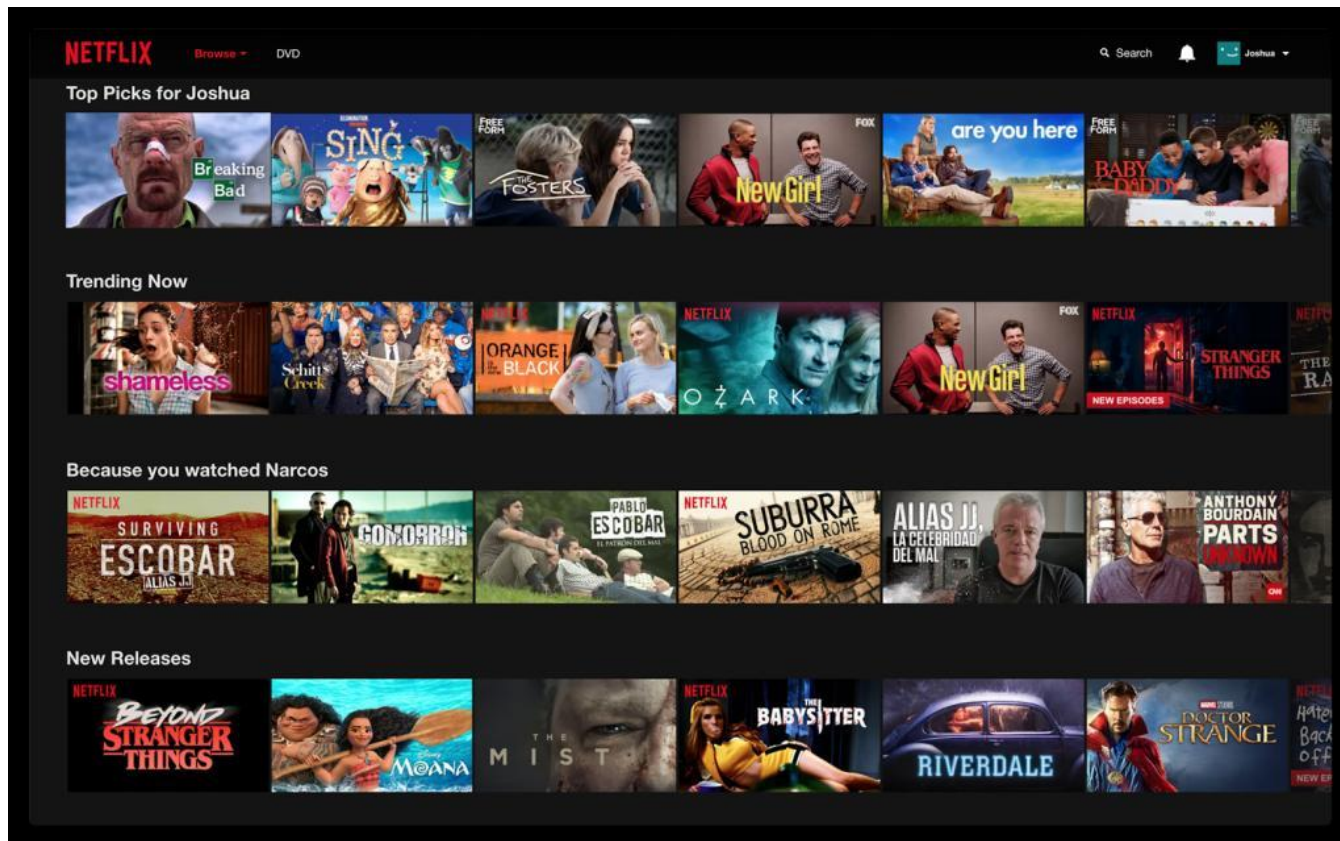


Lecture 15 – Reducing Dimensionality II



Today's Learning Outcomes

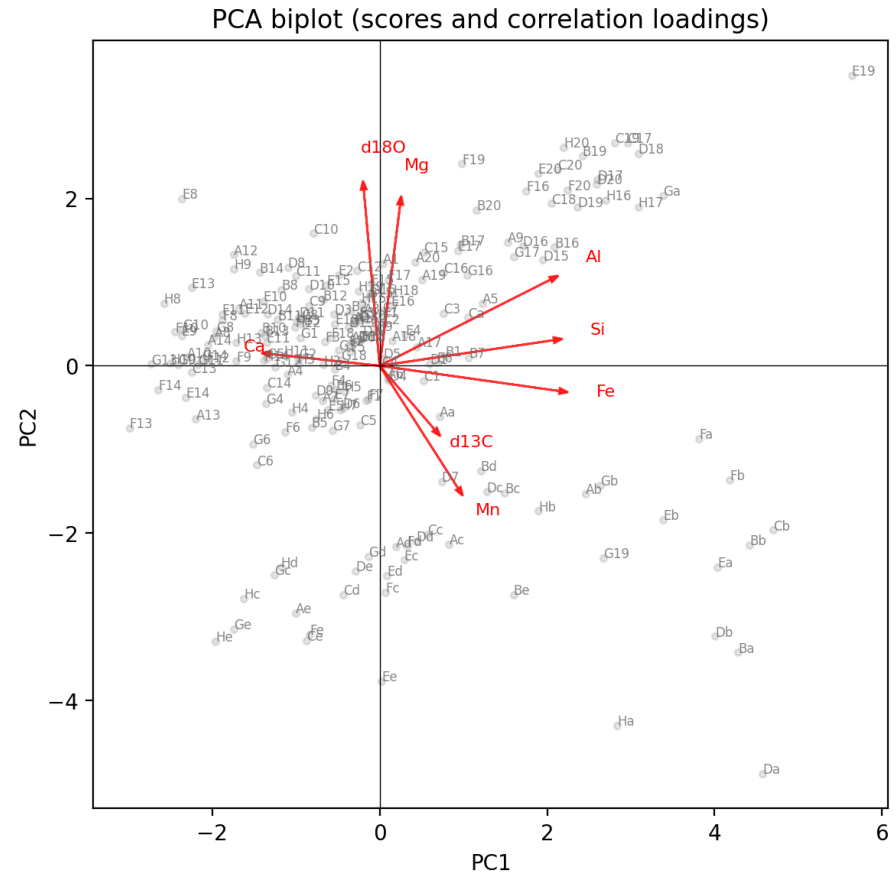
1. Know how the k-means approach to clustering works
2. Be able to explain the limitations and caveats of the method
3. Be able to run a k-means analysis in Python

Dimensionality & Clustering

- Last class we covered PCA analysis which takes a larger number of observations of variables and creates a new set of composite variables
 - Reduces the number of variables of interest
- Clustering approaches take in large amounts of observations are groups data points based on some similarity metric
 - Effectively reduces the “number” of data points

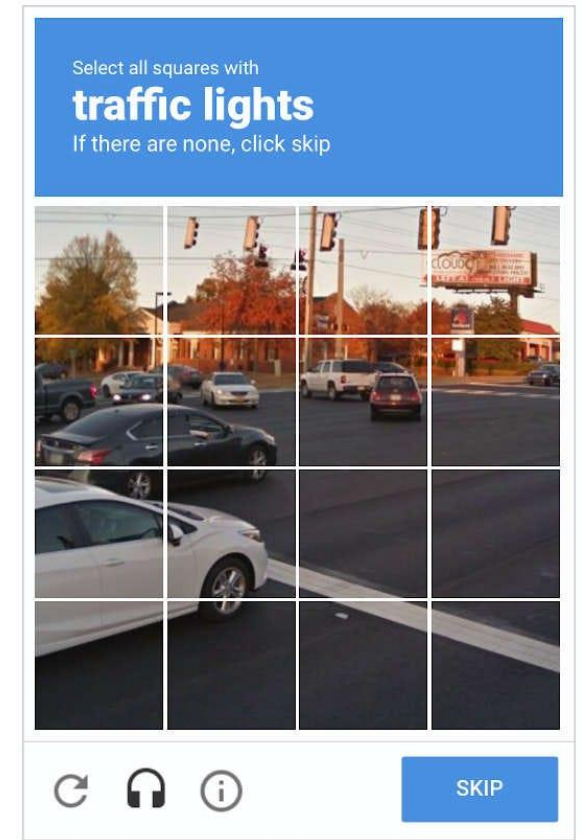
Exploratory Analyses

- PCA and the clustering approaches today are both exploratory, not explanatory
- Visualize and describe data but are not performed as a hypothesis test or explanatory model of why



Exploratory Analyses

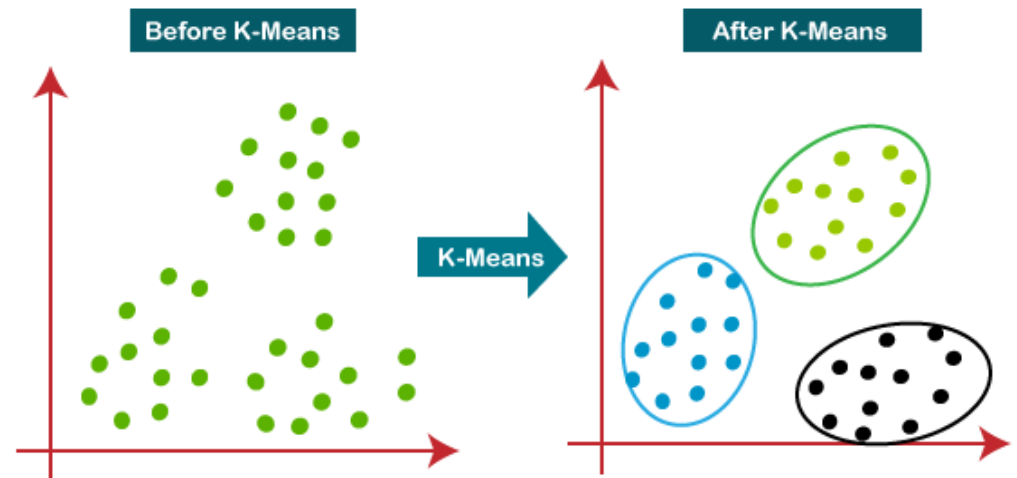
- PCA and cluster analyses today are unsupervised models
 - Not given any known outcomes
 - Purely data-driven and exploratory
- Supervised models are approaches which....
 - use a training set with known outcomes
 - build a model and predict future outcomes



You [helping self-driving car](#) AI via [supervised learning](#)

Clustering

- Idea is to take a dataset and split it into k clusters
 - Split data into categories for next decision in analysis
 - Identify natural breaks or boundary conditions
 - Recommendation/search engine results ([examples](#))
- For data with only a 2 or 3 axes plotting alone will usually spot groups
- With many axes clustering algorithms are critical



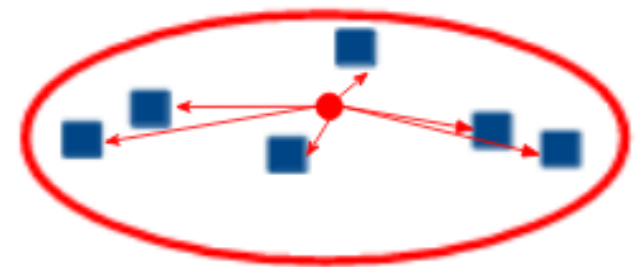
Clustering Optimization

Two potential goals when clustering...

1. Minimize differences within clusters
 - Points within clusters as similar as can be
2. Maximize differences between clusters
 - Points of each cluster are as different as possible from other clusters

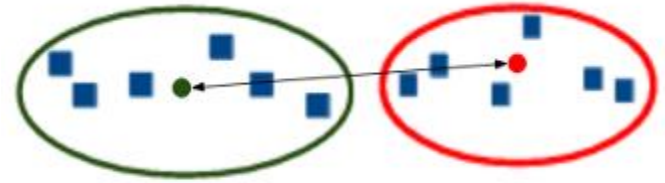
Clustering Mechanism (intra-cluster)

- Minimize sum of all distances of points to their centroid
- Sum of distances for all clusters is trying to be minimized
 - Looking for smallest value



Intra cluster distance

Clustering Mechanism (inter-cluster)



Inter cluster distance

- Consider maximizing distances between clusters
- Dunn Index =
$$\frac{\min(\text{inter-cluster distance})}{\max(\text{intra-cluster distance})}$$
- Higher value means larger distance between clusters and/or more tightly grouped clusters
 - Value < 1 would be where some clusters are closer to each other than the distance within at least one cluster

Measuring Similarity

Similarity can be measured in lots of different ways but calculations broadly fall into a few cases

Data is continuous (all values possible) and distances between values matter → use **Euclidean distances**

- Pythagorean theorem $\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$



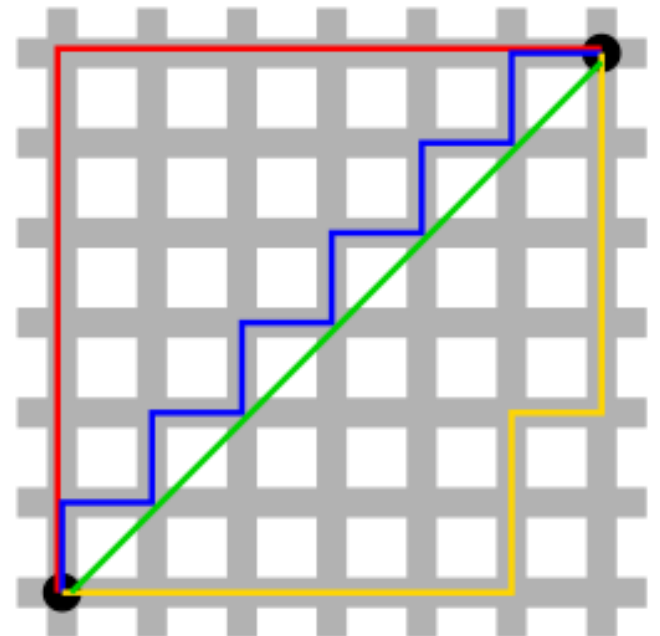
Measuring Similarity

Similarity can be measured in lots of different ways but calculations broadly fall into a few cases

Data is not continuous (discrete, binned, categorical, etc.) and distances between values is not meaningful/linear → use

Manhattan distances

- $|x_2 - x_1| + |y_2 - y_1|$



Measuring Similarity

Similarity can be measured in lots of different ways but calculations broadly fall into a few cases

Metric is correlation/match between data points → use **cosine similarity**

- $\cos(\theta) = \frac{(x \cdot y)}{(\|x\| \cdot \|y\|)}$
- $\cos(\theta)$ where theta is metric varies from -1 to 1 with 1 being a perfect match and -1 a direct opposite

This is used for most recommendation algorithms

Clustering Optimization

Two potential goals when clustering...

1. Minimize differences within clusters

- Points within clusters as similar as can be

2. Maximize differences between clusters

- Points of each cluster are as different as possible from other clusters

k-means technique we will focus on is concentrated on criteria 1 (not so much concern for criteria 2)

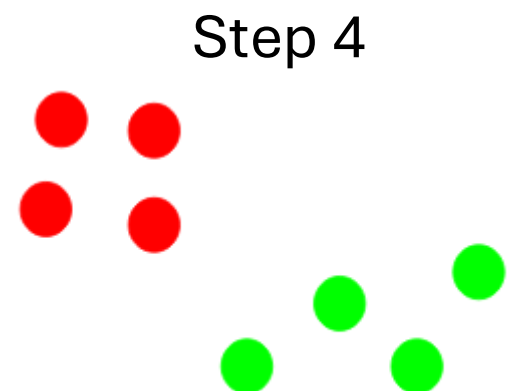
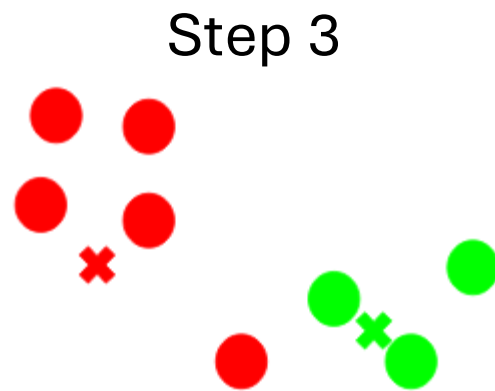
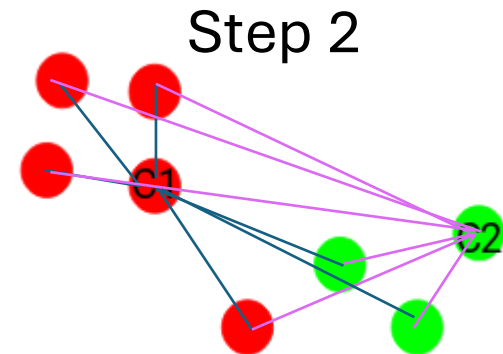
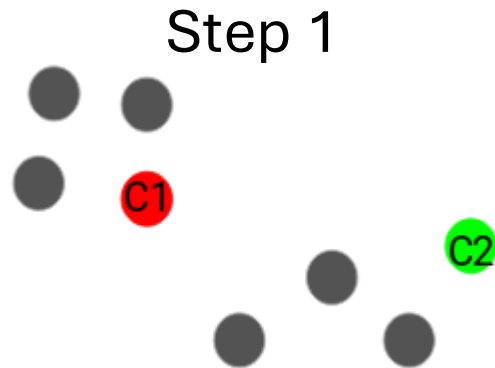
K-means clustering

Minimizes distance within clusters as optimization criteria (k is the number of clusters the data is being spit into)

Algorithm

1. Randomly select k points as the first centroids of clusters
2. Calculate each points distance the centroids and assign each point to the nearest one
3. Recalculate the centroid as the average position of cluster points
4. Repeat steps 2 and 3 until stability

K-means clustering



k-means (k=2)

Why K-means in Earth Sciences?

Earth science data is often high-dimensional — many variables measured per sample. K-means helps us find meaningful groups without a priori labels.

Petrology

Classify volcanic rocks by major-element geochemistry (SiO_2 , MgO , FeO , CaO ...)

Climatology

Group weather stations into climate zones by temperature & precipitation patterns

Seismology

Cluster seismic events by waveform features (P-wave arrival, magnitude, frequency)

Soil Science

Identify soil types from texture (% sand/silt/clay) and organic content

Remote Sensing

Segment satellite images into land-cover classes (ocean, forest, desert...)

Oceanography

Cluster ocean water masses by temperature, salinity, and dissolved oxygen

Example 1 — Climate Zone Classification

Cluster weather stations by long-term monthly climate data:

- Mean monthly temperature (12 values)
- Mean monthly precipitation (12 values)
- Seasonality index, frost days, humidity...

Each weather station = one row in dataset:

Station_ID | T_{Jan} | T_{Feb} | ... | T_{Dec} | P_{Jan} | P_{Feb} | ... | P_{Dec} | ...

Expected clusters:

Tropical

Hot year-round, high precipitation

Arid

Hot & dry, low seasonal variation

Temperate

Warm summers, mild winters, seasonal rain

Continental

Large temperature range, cold winters

Polar

Cold year-round, very low precipitation

Example 2 — Soil Type Classification

Soil profiles are described by many variables and k-means groups sites into coherent soil types:

- USDA soil texture triangle defines 12 classes based on sand/silt/clay
- K-means can discover these groupings automatically without the textbook diagram
- Can be extended to multi-depth profiles (3D clustering)
- Applications: precision agriculture, erosion risk mapping, carbon stock estimation

→ *Adding more variables (chemistry, biology) moves beyond the textbook triangle
— this is where clustering earns its keep!*

Soil Classification Variables

- % Sand
- % Silt
- % Clay
- Organic matter (%)
- pH
- Cation exchange capacity
- Bulk density (g/cm³)
- Hydraulic conductivity

Example 3 — Volcanic Rock Classification

Geochemical classification of volcanic rocks using major oxide measurements:

Basalt	Andesite	Rhyolite
SiO₂ ~48–52% MgO ~7–12% FeO ~8–12% CaO ~9–12% Oceanic crust, Hawaii, Iceland	SiO₂ ~57–63% MgO ~2–6% FeO ~5–9% CaO ~5–8% Subduction zones, Cascades, Andes	SiO₂ ~69–77% MgO ~0–2% FeO ~1–4% CaO ~0–2% Continental crust, Yellowstone caldera

Always standardize (z-score) geochemical data before k-means — oxide concentrations span very different ranges!

Steps for Performing K-means in Python

1. Load the data

```
df = pd.read_csv("combined.csv")
```

2. Select features for clustering

```
features = ["SiO2", "TiO2"]
```

```
X = df[features].values
```

3. Standardize the features

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

Steps for Performing K-means in Python

4: Run K-Means with k=3 (we know there are 3 rock types)

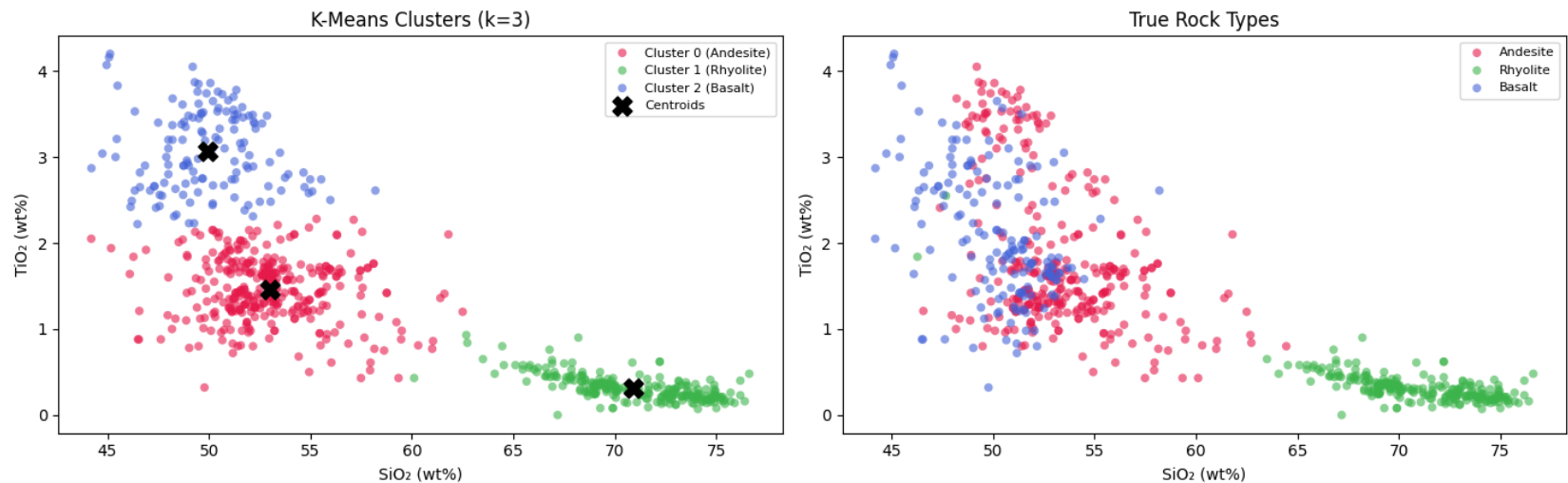
```
kmeans = KMeans(n_clusters=3, random_state=42,  
n_init=10)
```

```
kmeans.fit(X_scaled)
```

```
df["Cluster"] = kmeans.labels_
```

Steps for Performing K-means in Python

K-Means Clustering of Volcanic Rocks
Features: SiO₂ and TiO₂ only



Data source: <https://georoc.eu/georoc/new-start.asp>

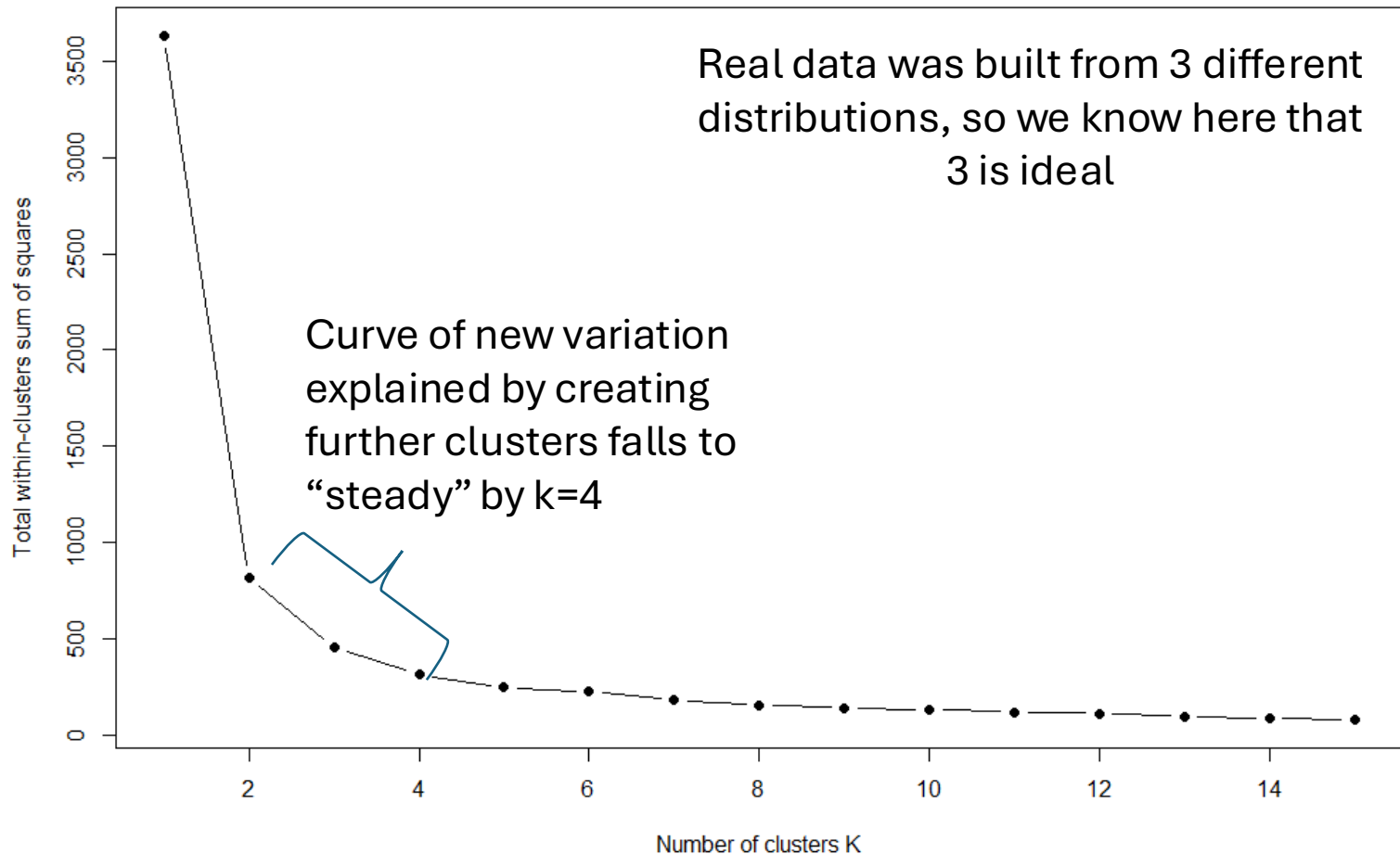
How to Pick Cluster Number

Since the k-means clustering approach is exploratory there's not always an obvious ideal number of clusters

Program will make as many clusters as you specify but clearly n groups of n data points is pointless

- Commonly used qualitative cutoff is the **Elbow Approach**
- Iteratively runs multiple cluster analyses with different k values, looks to identify where additional variation explained drops off

How to Pick Cluster Number



Challenge 1 — Sensitivity to Initialization

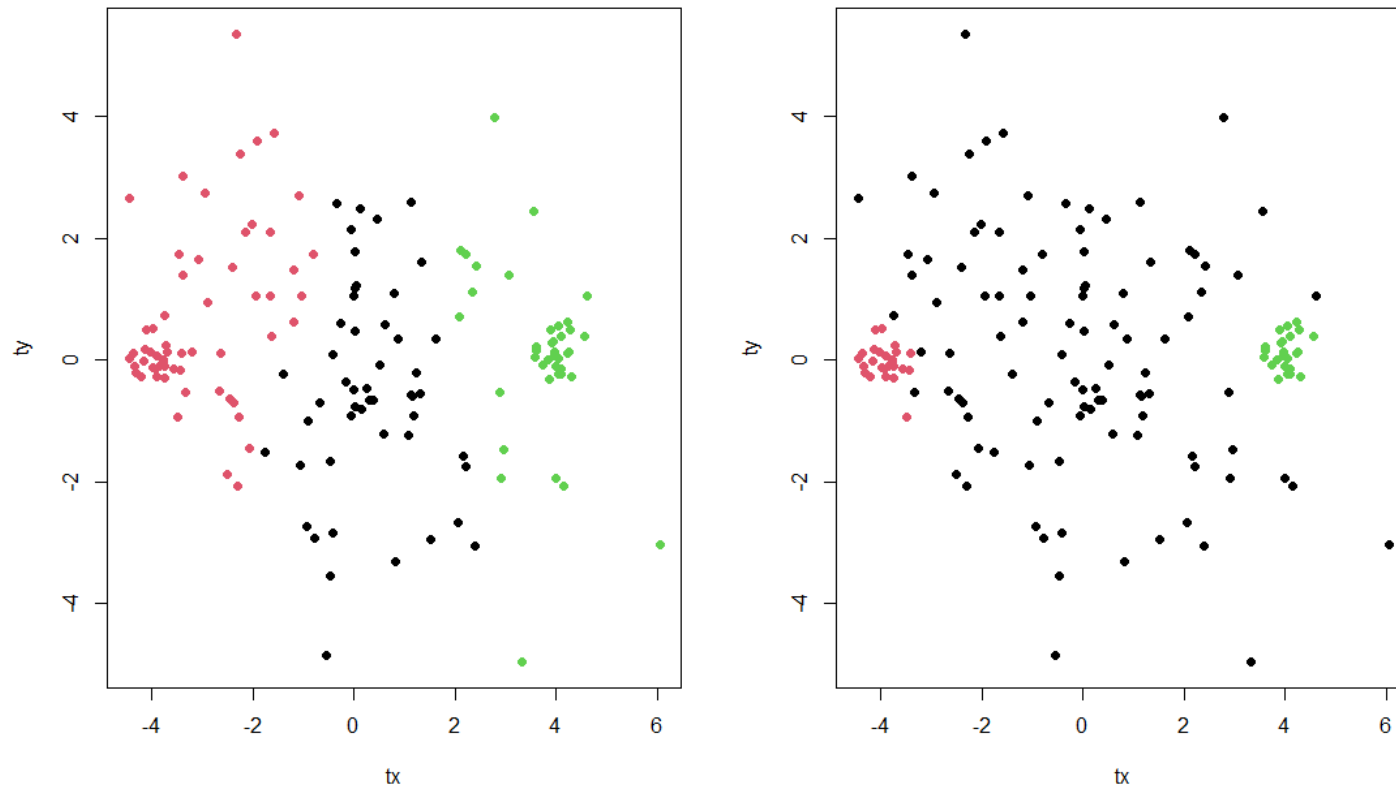
K-means converges to a local, not guaranteed global minimum.

- Different random starting centroids can produce different final clusters
- Bad initialization may lead to poor cluster structure
- Standard fix: run k-means many times (`n_init`) and keep the best WCSS result
- Better fix: K-Means++ initialization — places starting centroids far apart from each other

```
# Default: K-Means++ initialization (init='k-means++')
# n_init=10 runs the algorithm 10 times, keeps best result
kmeans = KMeans(
    n_clusters=3,
    init='k-means++',    # smart initialization
    n_init=10,           # 10 independent runs
    random_state=42       # for reproducibility
)
labels = kmeans.fit_predict(X_scaled)
```

Challenge 2 — Cluster Shape and Size

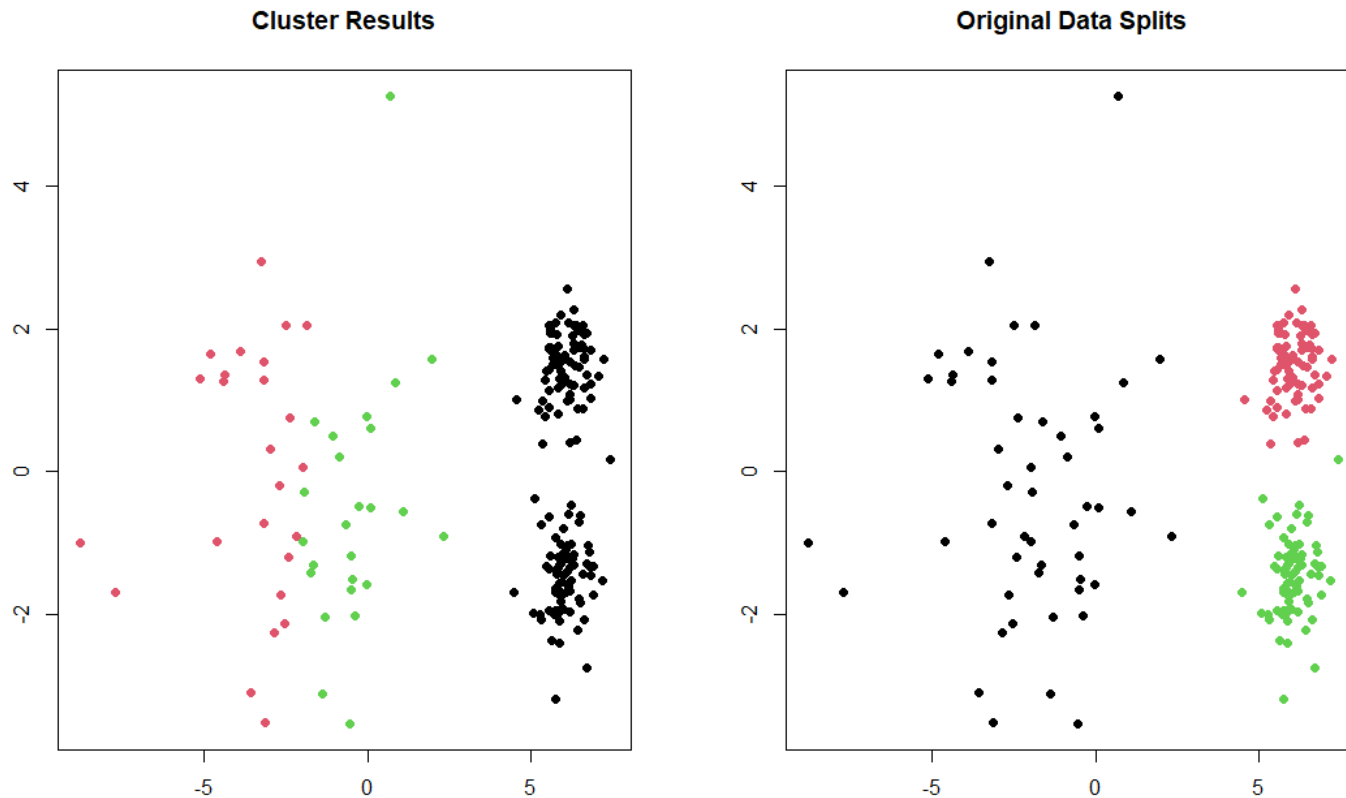
Does not guarantee equally-sized clusters (may or may not be a problem)



Example: unequal cluster sizes

Challenge 2 — Cluster Shape and Size

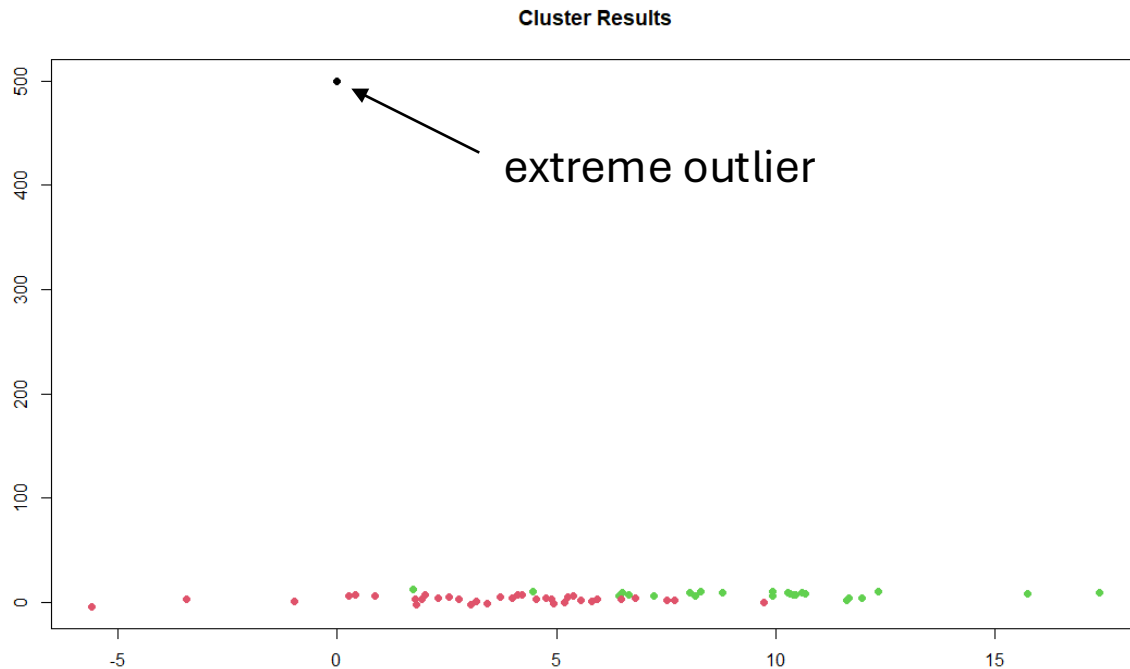
Cluster number can create odd categories if data not visualized appropriately (computer does what it's told)



Example: wrong choice of k forces odd artificial boundaries

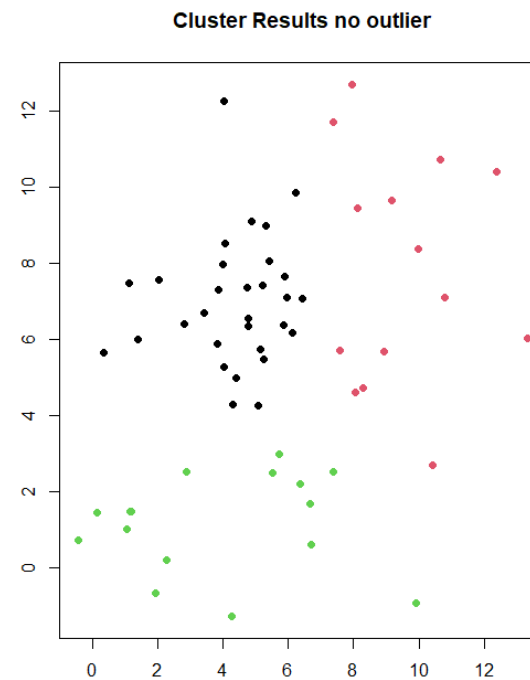
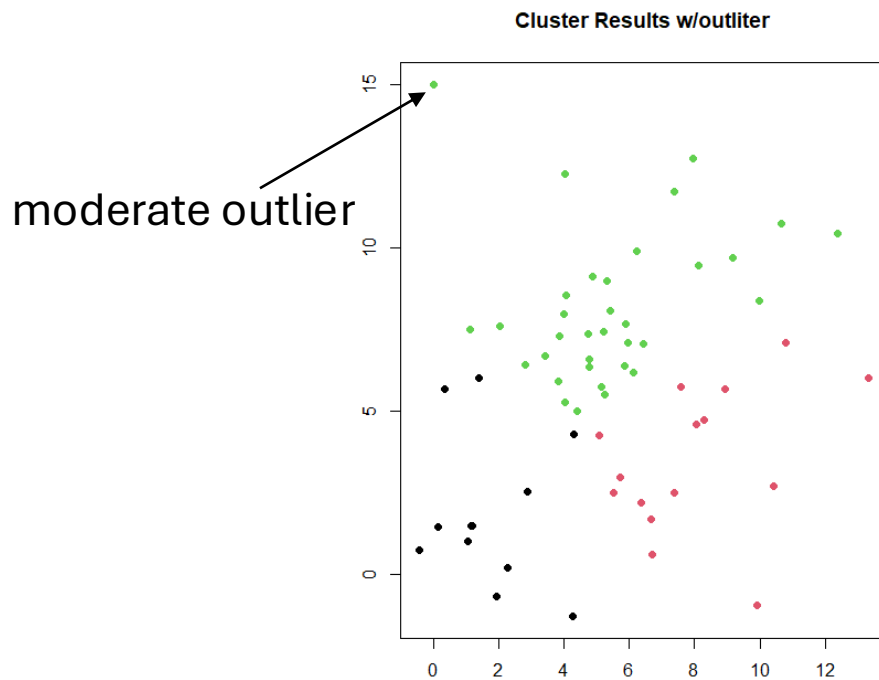
Challenge 3 — Outliers

Since it's a distance-based measure k-means can be vulnerable to outlier points (i.e. points add a huge amount of distance to whichever cluster and to centroid location)



Challenge 3 — Outliers

Since it's a distance-based measure k-means can be vulnerable to outlier points (i.e. points add a huge amount of distance to whichever cluster and to centroid location)



Challenge 4 — Feature Scaling

K-means uses Euclidean distance — variables with large numeric ranges dominate the clustering!

Geochemical example:

SiO₂ ranges 45–80% → distance contribution ~ 35 units

TiO₂ ranges 0.1–3% → distance contribution ~ 3 units

→ SiO₂ would dominate and TiO₂ would be almost ignored!

Fix: standardize (z-score) each feature before clustering:

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(df)    # zero mean, unit variance
```

Clustering Summary

All clustering does is group by some mathematical distance formula among points

- Any “distance” measure you can think up could be applied
- Can apply to PCA axes to get clusters of points based on the composite variables

Exploratory analysis whose final result can change based on random start points, should not be trusted as truth alone

Today's Learning Outcomes

1. Know how the k-means approach to clustering works
2. Be able to explain the limitations and caveats of the method
3. Be able to run a k-means analysis in Python